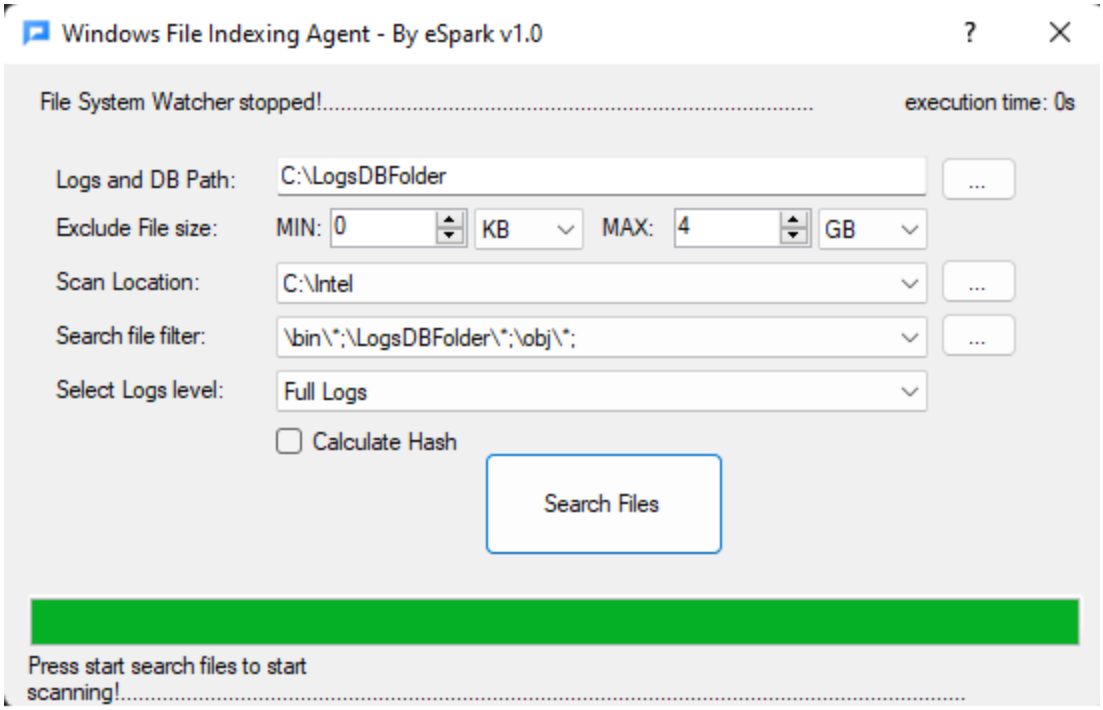


Windows File Indexing Agent

v01.00.00

—

By eSpark Consulting Group
Karachi, Pakistan

Overview

A small windows utility to store Folders/Files history, it Scan all the files in a user defined location and then its functionality for FileSystemWatcher will collect any changes if folder/file is Changed/Renamed/Created or Deleted in sqlite database. Any error occurred during operation is also logged using Best logging open source utility called NLog.

Goals

1. Overview The Windows File Indexing Agent is a software application designed to scan and index files on a Windows PC or server. It collects and stores metadata about each file, including filename, creation date, modification date, last accessed date, file size, and a SHA256 hash of the file contents.

2. Functional Requirements

   2.1 File Scanning The agent shall recursively scan all directories on specified drives or paths. The agent shall handle various file systems supported by Windows (NTFS, FAT32, exFAT). The agent shall be capable of scanning network drives when properly authenticated.

   2.2 Metadata Collection For each file encountered, the agent shall collect the following metadata: Full file path and filename File creation date and time File last modification date and time File last accessed date and time File size in bytes SHA256 hash of the file contents

   2.3 Indexing The agent shall store the collected metadata in a local database for quick searching and retrieval. The database schema should be optimized for fast querying based on any of the collected metadata fields.

   2.4 Scheduling The agent shall support scheduled scans (e.g., daily, weekly, monthly). The agent shall also support on-demand scans initiated by the user.

   2.5 Real-time Monitoring The agent shall implement a file system watcher to detect and index new or modified files in real-time.

3. Non-Functional Requirements

   3.1 Performance The agent shall be optimized for minimal system resource usage during scanning and indexing. The agent shall be capable of indexing at least 100,000 files per hour on a standard modern PC (Intel i5 or equivalent, 8GB RAM, SSD).

   3.2 Scalability The agent shall be able to handle file systems with millions of files. The database should be designed to efficiently manage large volumes of data.

   3.3 Security The agent shall run with minimal required permissions to perform its tasks. All stored data shall be encrypted at rest using industry-standard encryption methods.

3.4 Configurability The agent shall provide a configuration file to set scan locations, schedules, and other options. The agent shall support inclusion and exclusion rules based on file patterns, sizes, or locations.

3.5 Logging The agent shall maintain detailed logs of its operations, including scan times, and any errors encountered.

4. Technical Specifications

4.1 Development Environment Language: C# (.NET Framework 4.8 or .NET Core 3.1+) IDE: Visual Studio 2019 or later

4.2 Libraries and Components

- File System Access: System.IO namespace
- SHA256 Hashing: System.Security.Cryptography namespace
- Database: SQLite for local storage
- Scheduling: Windows Task Scheduler or Quartz.NET
- File System Watcher: FileSystemWatcher class
- Provide a system tray icon for quick access to agent functions.

4.3 Installation Develop an installer package (e.g., using WiX Toolset) for easy deployment. Include options for silent installation for enterprise environments.

5. 5. Testing Requirements

5.1 Unit Testing Implement unit tests for all core functions, including file metadata extraction, hashing, and database operations.

5.2 Integration Testing Test the agent's performance and accuracy on various Windows versions (Windows 10, Windows Server 2016/2019). Verify correct operation with different file systems and network drives.

5.3 Performance Testing Conduct stress tests with large file systems (1M+ files) to ensure scalability. Profile the application to identify and optimize any performance bottlenecks.

6. Documentation

6.1 User Manual Provide a comprehensive user manual detailing installation, configuration, and usage of the agent.

6.2 API Documentation If applicable, document any APIs for integrating the agent with other systems.

6.3 Developer Documentation Maintain inline code documentation and a separate developer guide for future maintenance and enhancements.

7. Maintenance and Support

   7.1 Updates Design the agent with an auto-update mechanism to easily deploy bug fixes and enhancements.

   7.2 Monitoring Implement error reporting to a central server for proactive issue detection and resolution.

8. Future Enhancements

   8.1 Once local capabilities are tested, data will be securely integrated with our cloud API to collect and process the telemetry.

Milestones

I. Functional Requirements

2.1 File Scanning The agent shall recursively scan all directories on specified drives or paths. The agent shall handle various file systems supported by Windows (NTFS, FAT32, exFAT). The agent shall be capable of scanning network drives when properly authenticated.

2.2 Metadata Collection For each file encountered, the agent shall collect the following metadata: Full file path and filename File creation date and time File last modification date and time File last accessed date and time File size in bytes SHA256 hash of the file contents

2.3 Indexing The agent shall store the collected metadata in a local database for quick searching and retrieval. The database schema should be optimized for fast querying based on any of the collected metadata fields.

2.4 Scheduling The agent shall support scheduled scans (e.g., daily, weekly, monthly). The agent shall also support on-demand scans initiated by the user.

2.5 Real-time Monitoring The agent shall implement a file system watcher to detect and index new or modified files in real-time.

II. Non-Functional Requirements

3.1 Performance The agent shall be optimized for minimal system resource usage during scanning and indexing. The agent shall be capable of indexing at least 100,000 files per hour on a standard modern PC (Intel i5 or equivalent, 8GB RAM, SSD).

3.2 Scalability The agent shall be able to handle file systems with millions of files. The database should be designed to efficiently manage large volumes of data.

3.3 Security The agent shall run with minimal required permissions to perform its tasks. All stored data shall be encrypted at rest using industry-standard encryption methods.

3.4 Configurability The agent shall provide a configuration file to set scan locations, schedules, and other options. The agent shall support inclusion and exclusion rules based on file patterns, sizes, or locations.

3.5 Logging The agent shall maintain detailed logs of its operations, including scan times, and any errors encountered.

III. Technical specifications

4.1 Development Environment Language: C# (.NET Framework 4.8 or .NET Core 3.1+) IDE: Visual Studio 2019 or later

4.2 Libraries and Components

- File System Access: System.IO namespace
- SHA256 Hashing: System.Security.Cryptography namespace
- Database: SQLite for local storage
- Scheduling: Windows Task Scheduler or Quartz.NET
- File System Watcher: FileSystemWatcher class
- Provide a system tray icon for quick access to agent functions.

4.3 Installation Develop an installer package (e.g., using WiX Toolset) for easy deployment. Include options for silent installation for enterprise environments.

IV. Testing Requirements

5.1 Unit Testing Implement unit tests for all core functions, including file metadata extraction, hashing, and database operations.

5.2 Integration Testing Test the agent's performance and accuracy on various Windows versions (Windows 10, Windows Server 2016/2019). Verify correct operation with different file systems and network drives.

5.3 Performance Testing Conduct stress tests with large file systems (1M+ files) to ensure scalability. Profile the application to identify and optimize any performance bottlenecks.

V. Documentation

6.1 User Manual Provide a comprehensive user manual detailing installation, configuration, and usage of the agent.

VI. Maintenance and Reports/Logs

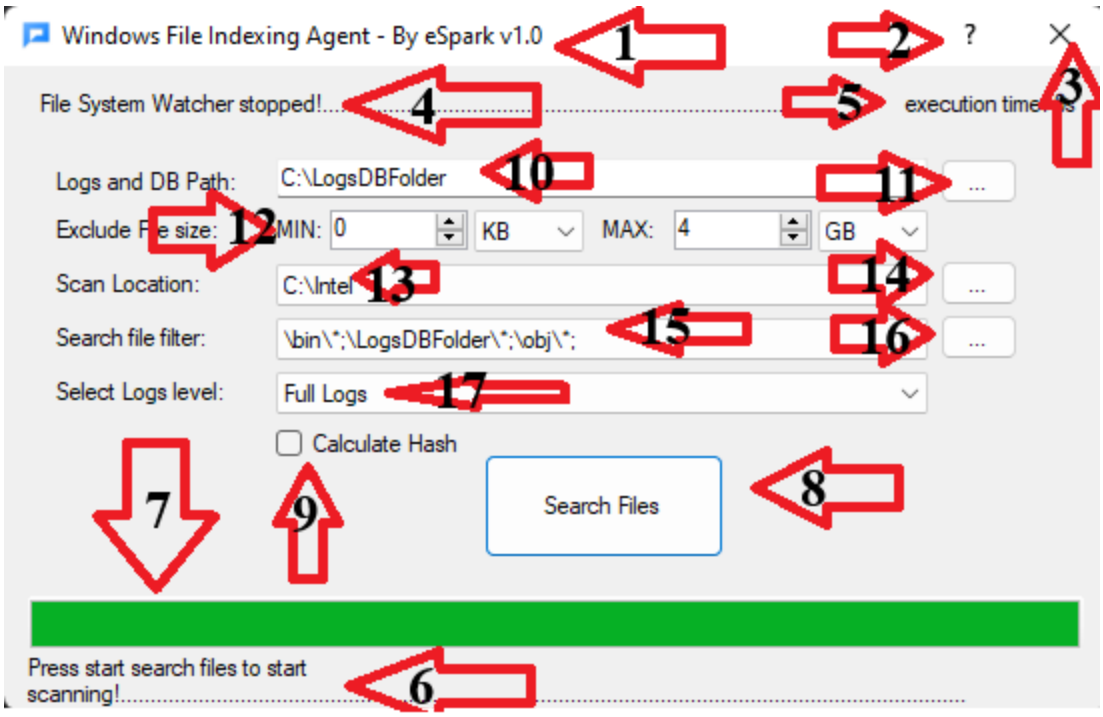
7.1 Updates Design the agent with an auto-update mechanism to easily deploy bug fixes and enhancements.

7.2 Monitoring Implement error reporting to a central server for proactive issue detection and resolution.

Specifications

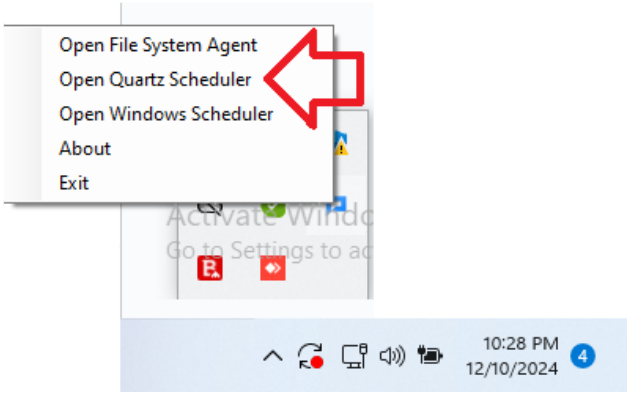
Main Dashboard/ Program Parameters

Image#1a



Image#1b





1# Product Title with program Version number displayed here.

2# This Help / Program documentation will be displayed.

3# Click to minimize in system tray [Please check system tray settings if it is not displayed, On some systems you will need to enable to show the icon and popup messages], Please follow Image#1b,

4# Here File system watcher status will be displayed

5# Here process execution time will be displayed, it will only show when Search button [#8] is pressed

6# When the process in running state all the current scanning files will be displayed, and after it completes it will show “Process completed successfully” message

7# Progress bar to show the remaining time of total process

8# Search button will start the search process will all the parameters that are selected by user, explained below,

9# When checked it will calculate the hash using SHA256 crypto Algorithm, [Please use when required as it will consume at least double of normal time]

10# Logs and DB path shows the selected path of Database and Logs to be stored when process runs, [It is Must Required first time only to give its path using button on 11# and second time all the parameters are stored in application already]

11# Please select the Logs file location to store for Errors/Warnings and Information for running process and also where the Database will be created to store Files/Folders information

12# Exclude file with Minimum and Maximum size, enter numeric value for Minimum and Maximum size and then select the file unit in KB=KiloByte, MB= MegaByte,GB=GigaByte

13# Scan Location is the required parameter to store the scanning locations for process to run, you can select any location using Button 14# and if the location you already selected and then press the Button 14# then it will be overwritten

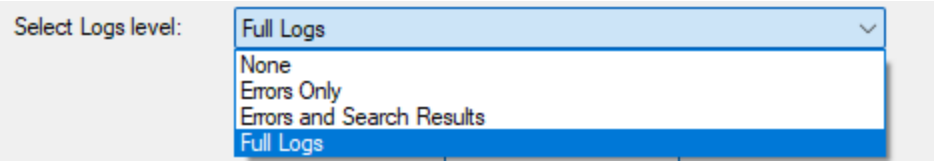
14# Browse the ScanLocation path

15# Search filter is used to exclude the folder location, [you must have to Add the folder that have been selected in #10 ie LogsAndDB path because logs are created continuously by this program and FileSystem watcher can deadlock the process] also you can add any number of folders,

16# The folder that needs to be excluded must follow pattern “\obj*;” for example, if “obj” folder in any subfolder exist the it will be skipped, use as many as you

required in one record, you can add any number of records, if you already selected some record and press button then that record will be overwritten.

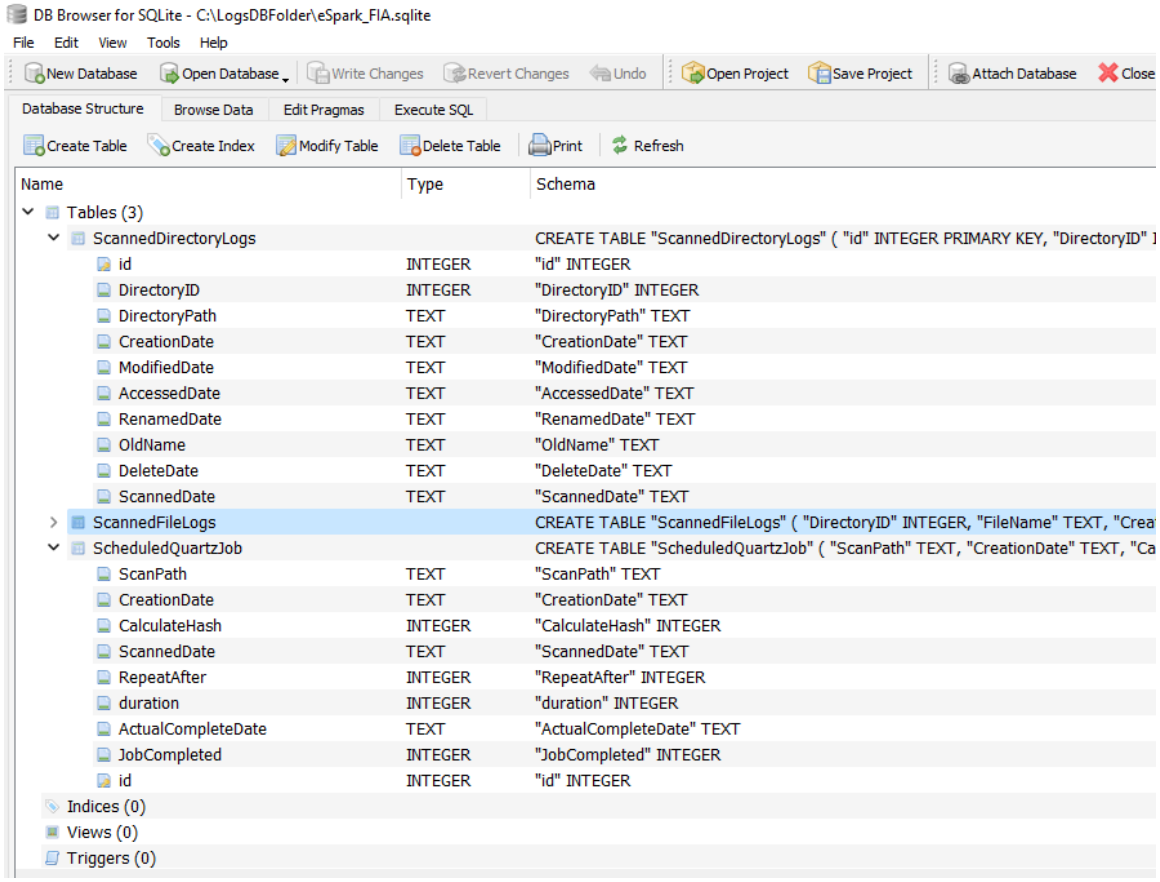
17# Select the Level of logs you need to create when program runs, [Please use with caution as more depth of logs are created the slower will be the program - use only when it is required]



Use None every time, else use Errors Only to get a detailed info for any error

Database and Logs

Image#2a

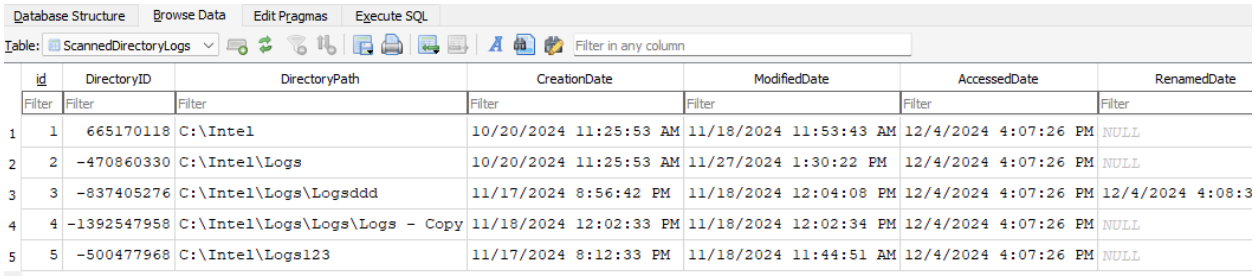


A Database has three tables ie for Folders and their Files and then Scheduled Jobs

Image#2b

Folder table contains DirectoryID and Path and other parameters

Each file in directory will use the DirectoryID

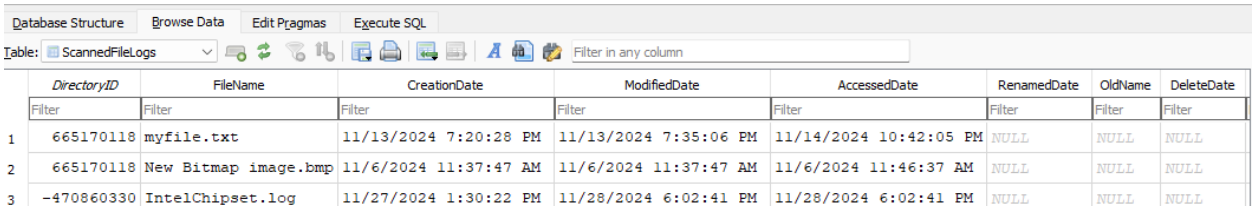


	id	DirectoryID	DirectoryPath	CreationDate	ModifiedDate	AccessedDate	RenamedDate
	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1	665170118	C:\Intel	10/20/2024 11:25:53 AM	11/18/2024 11:53:43 AM	12/4/2024 4:07:26 PM	NULL
2	2	-470860330	C:\Intel\Logs	10/20/2024 11:25:53 AM	11/27/2024 1:30:22 PM	12/4/2024 4:07:26 PM	NULL
3	3	-837405276	C:\Intel\Logs\Logsddd	11/17/2024 8:56:42 PM	11/18/2024 12:04:08 PM	12/4/2024 4:07:26 PM	12/4/2024 4:08:3
4	4	-1392547958	C:\Intel\Logs\Logs\Logs - Copy	11/18/2024 12:02:33 PM	11/18/2024 12:02:34 PM	12/4/2024 4:07:26 PM	NULL
5	5	-500477968	C:\Intel\Logs123	11/17/2024 8:12:33 PM	11/18/2024 11:44:51 AM	12/4/2024 4:07:26 PM	NULL

Image#2c

File table contains DirectoryID reference from ScannedDirectoryLogs table and file Name and other parameters

Each file in here use the DirectoryID reference



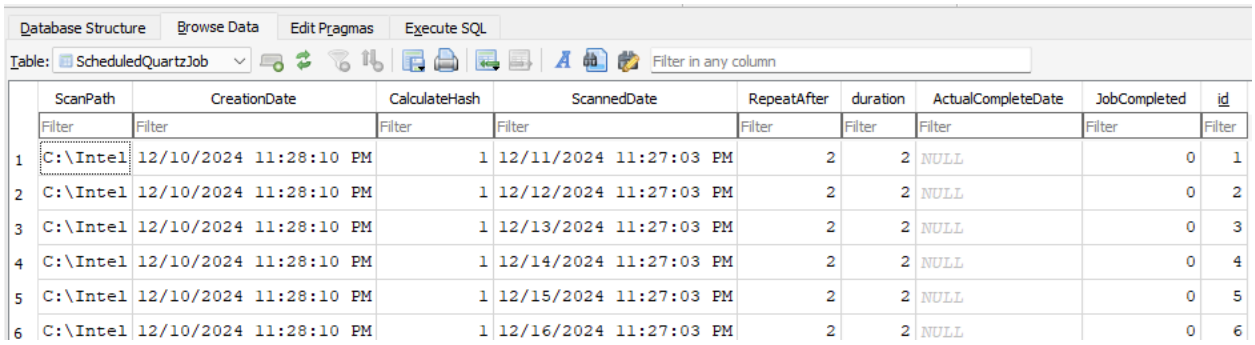
Database Structure | Browse Data | Edit Pragmas | Execute SQL

Table: ScannedFileLogs | Filter in any column

	DirectoryID	FileName	CreationDate	ModifiedDate	AccessedDate	RenamedDate	OldName	DeleteDate
	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	665170118	myfile.txt	11/13/2024 7:20:28 PM	11/13/2024 7:35:06 PM	11/14/2024 10:42:05 PM	NULL	NULL	NULL
2	665170118	New Bitmap image.bmp	11/6/2024 11:37:47 AM	11/6/2024 11:37:47 AM	11/6/2024 11:46:37 AM	NULL	NULL	NULL
3	-470860330	IntelChipset.log	11/27/2024 1:30:22 PM	11/28/2024 6:02:41 PM	11/28/2024 6:02:41 PM	NULL	NULL	NULL

Image#2d

All the jobs created by QuartzScheduler will be displayed here with CreationDate and ScanningDate



Database Structure | Browse Data | Edit Pragmas | Execute SQL

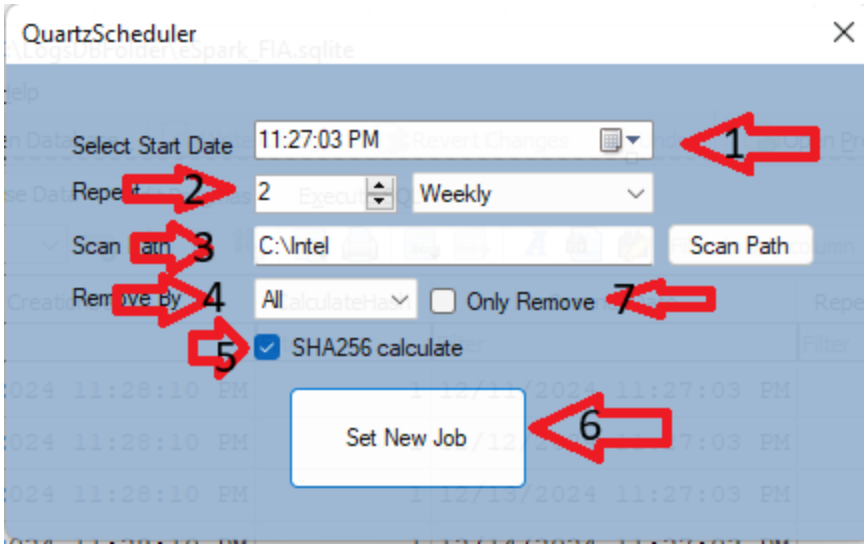
Table: ScheduledQuartzJob | Filter in any column

	ScanPath	CreationDate	CalculateHash	ScannedDate	RepeatAfter	duration	ActualCompleteDate	JobCompleted	id
	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	C:\Intel	12/10/2024 11:28:10 PM	1	12/11/2024 11:27:03 PM	2	2	NULL	0	1
2	C:\Intel	12/10/2024 11:28:10 PM	1	12/12/2024 11:27:03 PM	2	2	NULL	0	2
3	C:\Intel	12/10/2024 11:28:10 PM	1	12/13/2024 11:27:03 PM	2	2	NULL	0	3
4	C:\Intel	12/10/2024 11:28:10 PM	1	12/14/2024 11:27:03 PM	2	2	NULL	0	4
5	C:\Intel	12/10/2024 11:28:10 PM	1	12/15/2024 11:27:03 PM	2	2	NULL	0	5
6	C:\Intel	12/10/2024 11:28:10 PM	1	12/16/2024 11:27:03 PM	2	2	NULL	0	6

Scheduler using QuartzScheduler

To use QuartzScheduler Run the program and go to taskbar as shown in image and **Image#1b** right click on it you will get displayed some options, you can select "Open Quartz Scheduler" and then a window will be opened

Image#2da



1# Select the Start Time and Date where first Job will be started from this date time

2# Repeat interval with Never,Daily,Weekly and Monthly options,

If you select 2 and Weekly option the 2 weeks scheduled jobs will be created with equal time 24hrs time gaps for each day, ie total 14 jobs will be created

If you select 2 and Days option then only 2 jobs will be created

Same as for month

But Never option is used when you just need to clear records or clean table, or you can say as a dummy option

3# Which folder you need to scan when job is scheduled to start, Note for multiple folder you will need to create now jobs

4# Remove by option lets you remove jobs in two ways ie "Scan Path" when selected then only the path selected in 3# jobs will be removed including all scandate it is found, and option ALL when selected will remove ALL jobs when clear the table and then created new records HERE 7# when checked it will only remove the data and do not add records, this functionality is added only for debug the database

5# When checked it will calculate the hash using SHA256 crypto Algorithm, [Please use when required as it will consume at least double of normal time]

6# Press this button to create the jobs with all the parameters defined by user

Interesting Facts

Fact#1

The program is used to run as backend process and when you click close button as in image#1a point 3# then it will minimize to system tray,

To show it again you need to click the icon

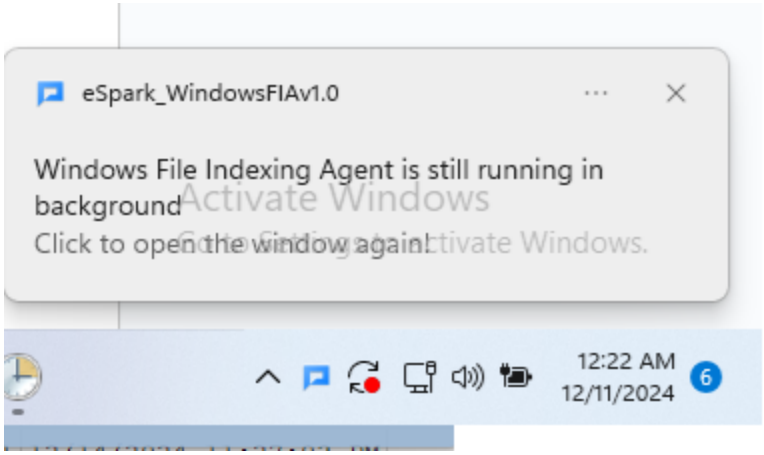
Fact#2

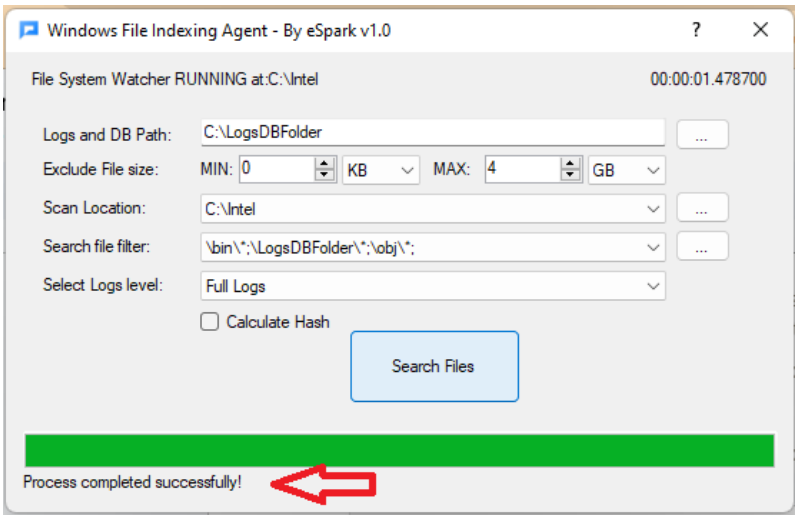
The program is used to run as backend process and when you click close button as in image#1a point 3# then it will minimize to system tray,

But to exit the program you will need to right click “Exit” as in image#1b

Fact#3

When the program is minimized it will show popup





Fact#4

When the process is completed successfully it will display “Process completed successfully”